

Image processing with multiprocessing using Python

Xiaoyuan Suo

Image processing using Python

- What is Image Processing?
 - A method to perform operations on images to enhance, analyze, or extract useful information.
 - Applications include medical imaging, computer vision, remote sensing, and digital photography.
- Basic Tasks
 - Enhancement – improve image quality (e.g., noise reduction, sharpening).
 - Segmentation – divide image into meaningful regions.
 - Feature Extraction – detect edges, shapes, or patterns.

Examples



Parallel Processing

- Why Parallel Processing?
 - Images are made of millions of pixels → processing is computationally intensive.
 - Many operations (e.g., filtering, convolution, pixel transformations) can be done independently on different pixels or regions.
 - Multi-core C++ implementations (using OpenMP, MPI, or CUDA) speed up computation and allow real-time processing.

Introduction to OpenCV

- What is OpenCV?
 - *Open Source Computer Vision Library* (OpenCV) is a popular open-source toolkit for image processing, computer vision, and machine learning.
 - Written in C++, with bindings for Python, Java, and others.
- Key Features
 - Image and video I/O and manipulation
 - Filters, transformations, and edge detection
 - Object, face, and motion detection
 - Integration with deep learning frameworks (TensorFlow, PyTorch)
 - Real-time computer vision applications
- Why Use OpenCV?
 - Highly optimized for speed and multi-core performance
 - Works across platforms (Windows, macOS, Linux)
 - Easy to integrate with C++ or Python projects

Examples of parallelizing image processing

```
GreyScale parallel.py - C:/Users/mshgo/Downloads/GreyScale parallel.py (3.11.9)
File Edit Format Run Options Window Help

import cv2
import numpy as np
from multiprocessing import Pool, cpu_count

def to_grayscale(section):
    """Convert a section of the image to grayscale."""
    return cv2.cvtColor(section, cv2.COLOR_BGR2GRAY)

def parallel_grayscale(image_path, num_workers=None):
    # Read the image
    image = cv2.imread(image_path)
    cv2.imshow("Grayscale Image (Parallel)", image)
    if image is None:
        print("Error: Could not load image.")
        return None

    # Determine number of processes
    if num_workers is None:
        num_workers = cpu_count()

    # Split the image into sections (rows)
    h = image.shape[0]
    sections = np.array_split(image, num_workers, axis=0)

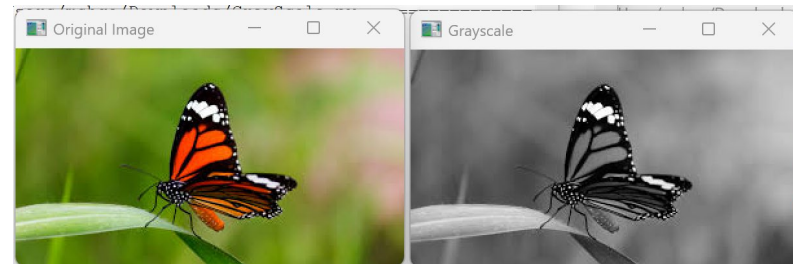
    # Parallel processing
    with Pool(processes=num_workers) as pool:
        gray_sections = pool.map(to_grayscale, sections)

    # Recombine the sections
    gray_image = np.vstack(gray_sections)
    return gray_image

if __name__ == "__main__":
    input_image = "input.jpg"
    output_image = "gray_parallel.jpg"
    image = cv2.imread(input_image)
    cv2.imshow("Grayscale Image (Parallel)", image)
    gray = parallel_grayscale(input_image)
    if gray is not None:
        cv2.imwrite(output_image, gray)
        cv2.imshow("Grayscale Image (Parallel)", gray)
        cv2.waitKey(0)
        cv2.destroyAllWindows()
```

```
GreyScale.py - C:/Users/mshgo/Downloads/GreyScale.py (3.11.9)
File Edit Format Run Options Window Help

import cv2
image = cv2.imread('input.jpg')
cv2.imshow('Original Image', image)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
cv2.imshow('Grayscale', gray)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Vectorized Operations in OpenCV

- OpenCV uses vectorized operations—performing computations on entire arrays (matrices) at once instead of looping pixel-by-pixel in Python.
- Why it matters:
 - Python loops are slow because they execute pixel operations one at a time.
 - OpenCV functions are written in optimized C/C++, using:
 - SIMD instructions (Single Instruction, Multiple Data)
 - Multithreading (via OpenMP or TBB)
 - This makes operations like filtering, addition, and color conversion orders of magnitude faster.

How to parallelize the process



Worker #1



Worker #2



Worker #3



Worker #4



Worker #5




```
import cv2 #opencv is already vectorized
#pip install opencv-python
import numpy as np

# Load input image
image = cv2.imread("input.jpg")

# Convert to HSV (Hue, Saturation, Value) color space
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

# Define range for green color in HSV
lower_green = np.array([35, 40, 40]) # adjust as needed
upper_green = np.array([85, 255, 255])

# Create mask where green pixels are 1
mask = cv2.inRange(hsv, lower_green, upper_green)

# Invert mask to keep non-green regions
mask_inv = cv2.bitwise_not(mask)

# Apply mask to keep only foreground
result = cv2.bitwise_and(image, image, mask=mask_inv)

# Save and display
cv2.imwrite("output.png", result)

cv2.imshow("Original", image)
cv2.imshow("No Green Background", result)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
import cv2 #pip install opencv-python
import numpy as np
import multiprocessing as mp

# Define green screen removal function
def remove_green(chunk):
    """Process a chunk of the image to remove green background."""
    hsv = cv2.cvtColor(chunk, cv2.COLOR_BGR2HSV)
    lower_green = np.array([35, 40, 40])
    upper_green = np.array([85, 255, 255])
    mask = cv2.inRange(hsv, lower_green, upper_green)
    mask_inv = cv2.bitwise_not(mask)
    result = cv2.bitwise_and(chunk, chunk, mask=mask_inv)
    return result

def parallel_remove_green(image, num_workers=4):
    """Split image into vertical chunks and process in parallel."""
    height = 0
    chunk_size = 1
    if image is not None:
        height = image.shape[0]
        chunk_size = height // num_workers

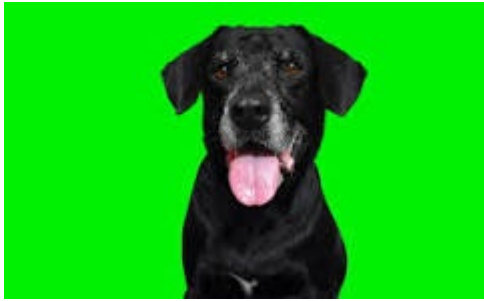
    # Split into chunks
    chunks = [image[i*chunk_size:(i+1)*chunk_size] for i in range(num_workers)]

    # Use multiprocessing pool
    with mp.Pool(processes=num_workers) as pool:
        results = pool.map(remove_green, chunks)

    # Concatenate back into one image
    return np.vstack(results)

if __name__ == "__main__":
    image = cv2.imread("input.jpg")
    result = parallel_remove_green(image, num_workers=mp.cpu_count())

    # Save and show
    cv2.imwrite("output_parallel.png", result)
    cv2.imshow("Original", image)
    cv2.imshow("No Green Background (Parallel)", result)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```



Homework: parallelize the image blur algorithm

```
blur.py - C:/Users/mshgo/Downloads/blur.py (3.11.9)
File Edit Format Run Options Window Help
import cv2

# Read the image
image = cv2.imread('input.jpg') # Replace with your image filename

# Check if the image was loaded successfully
if image is None:
    print("Error: Image not found.")
    exit()

# Apply Gaussian blur
blurred = cv2.GaussianBlur(image, (15, 15), 0)

# Display the original and blurred images
cv2.imshow('Original Image', image)
cv2.imshow('Blurred Image', blurred)

# Save the blurred image
cv2.imwrite('blurred_output.jpg', blurred)

# Wait until a key is pressed, then close windows
cv2.waitKey(0)
cv2.destroyAllWindows()
```

